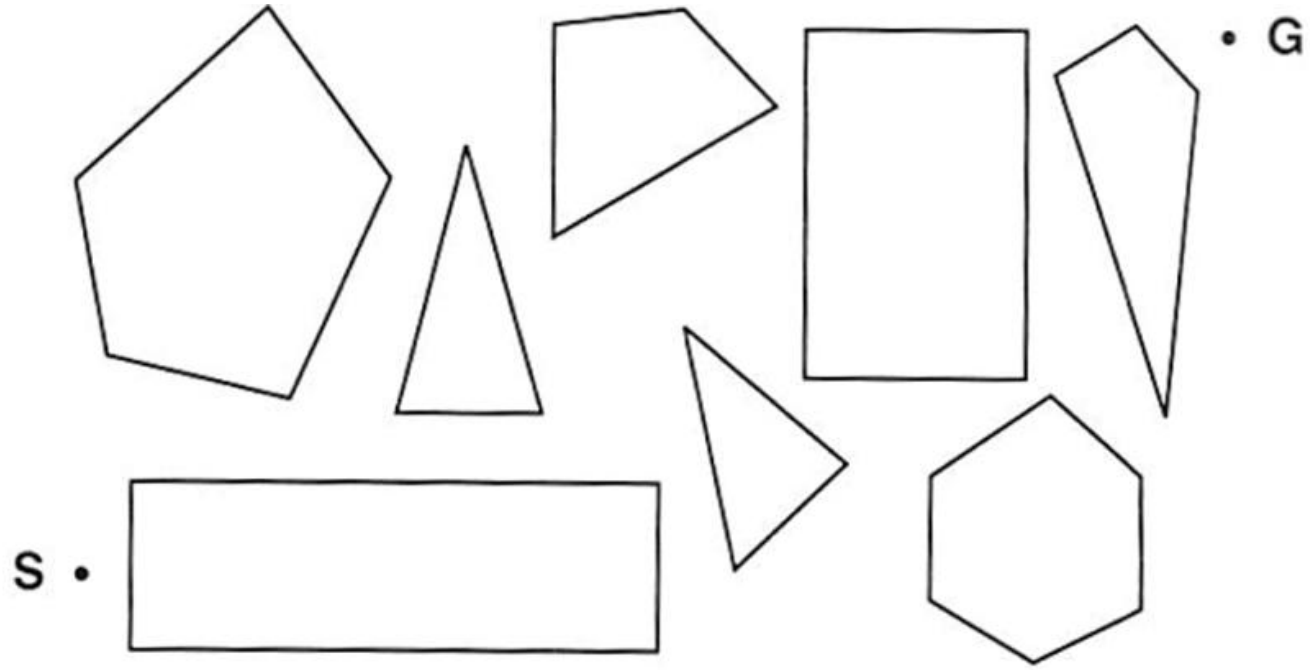
ARTIFICIAL INTELLIGENCE #1

by Elvis Avduli, E22001

1: State Space



State Space: The state space in this problem consists of the vertices of the polygons shown in the diagram. These vertices represent the possible positions that can be part of a path. Each state is modeled as a vertex of the graph formed by connecting these vertices (based on the edges or straight-line connections between them). A state is defined by the current vertex occupied and the set of its neighboring vertices to which one can move next.

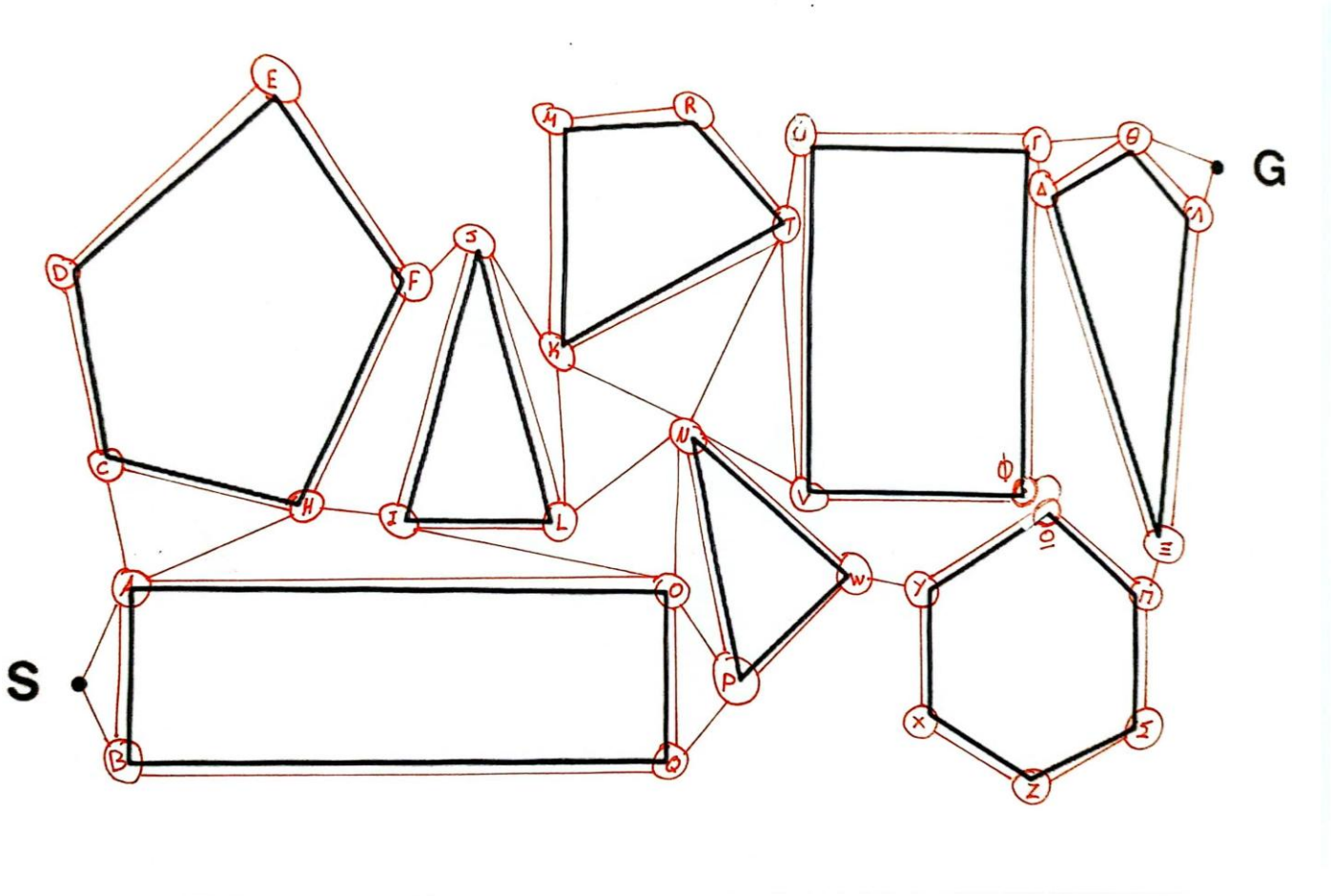
Successor Function: The successor function determines all neighboring vertices that are directly reachable from the current vertex. Two vertices are considered neighbors if they are connected by a straight line (i.e., if there is an edge linking the two vertices). This ensures that transitions are only valid if they follow a direct path between the vertices without crossing the interior of any polygon.

State Space Size: The size of the state space equals the total number of vertices across all polygons. Based on the diagram, the state space contains 35 states.

Shortest Path Characteristics: The shortest path between two vertices always consists of straight-line segments connecting the vertices of the polygons. This is because a straight-line connection between two vertices (if valid and does not intersect the interior of any polygon) is

geometrically the shortest possible path. Any attempt to pass through the surfaces of the polygons or to take a detour would result in a longer or invalid path. Hence, the solution adheres to the geometry of the graph defined by the polygon vertices.

A simple visualization of the method mentioned above could be something like this:



2.Search

2.A. The search problem

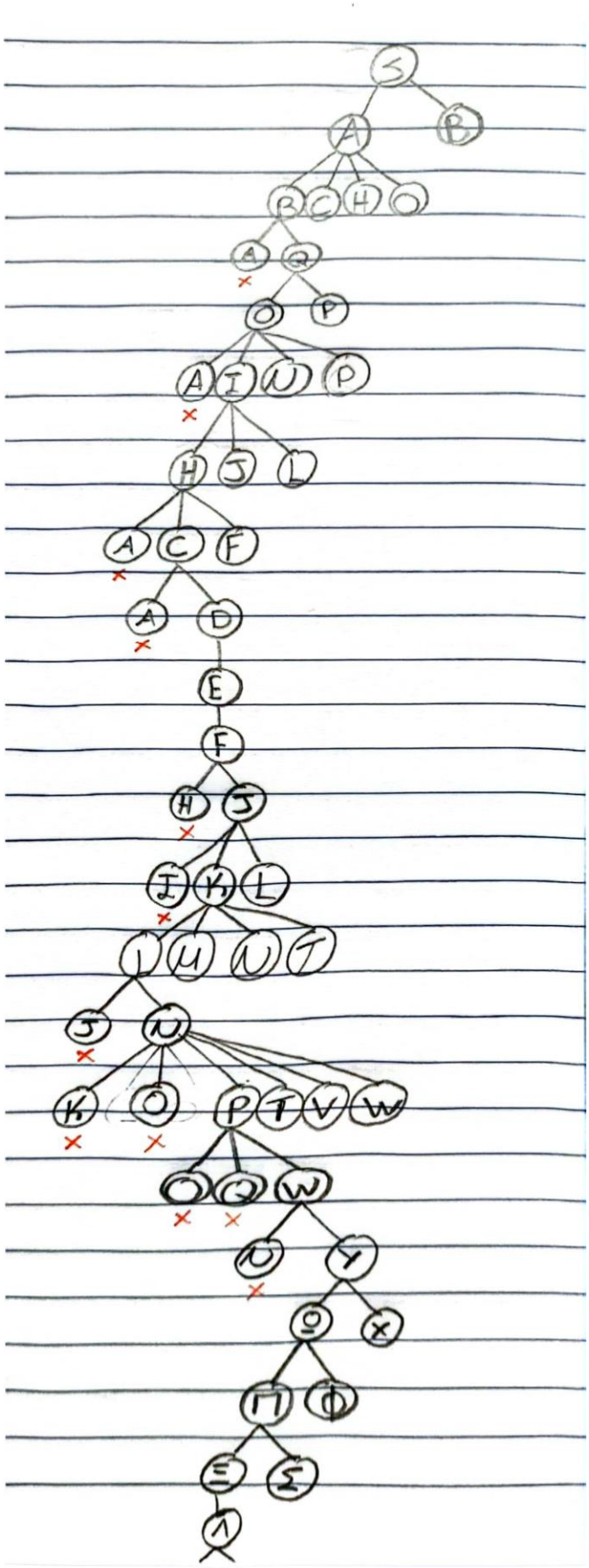
Graph search is a fundamental problem focusing on exploring a graph (a structure composed of nodes (vertices) and edges) to identify the best path, shortest route, or specific patterns within the graph. Nodes represent entities such as locations in a city network, users in a social network, or states in a machine learning problem. Edges connect nodes and can be directed or undirected, weighted or unweighted, representing relationships or transitions. Weights typically signify costs, distances, or time required to move from one node to another.

2.A.a. DFS of the problem above

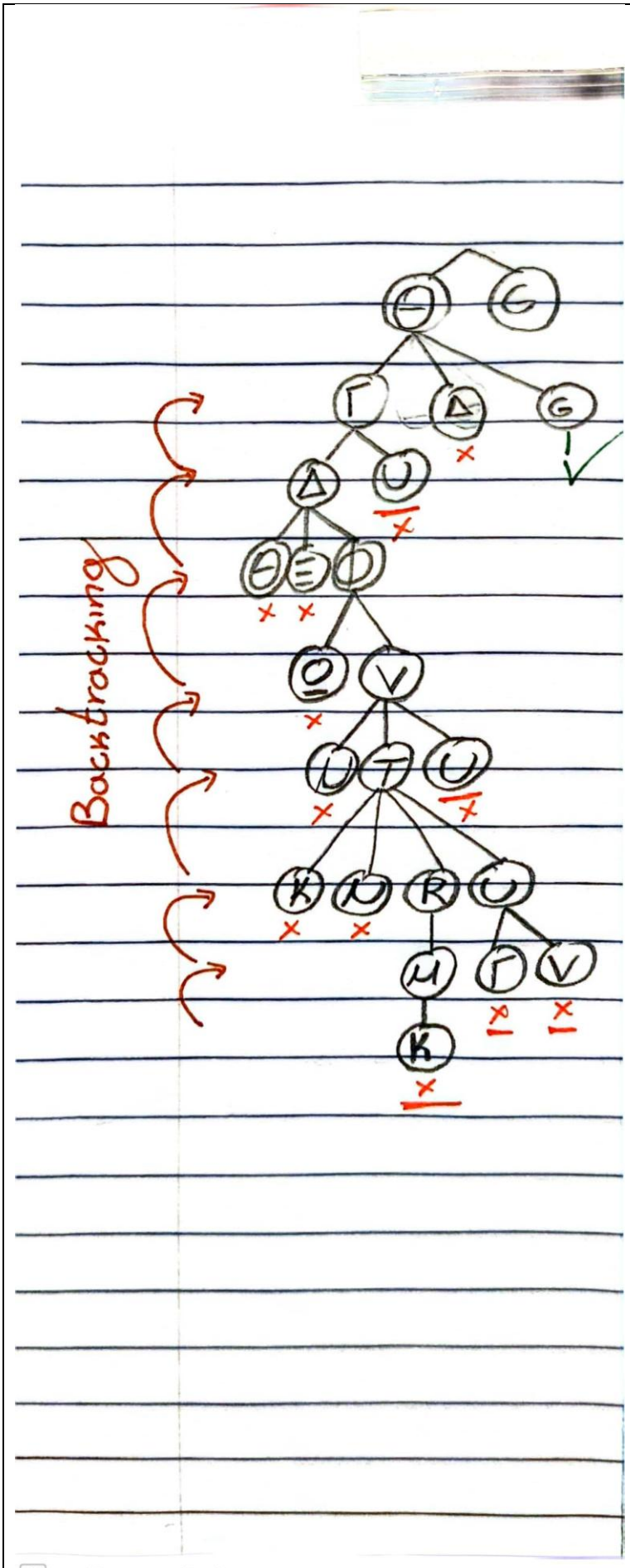
In the Depth-First Search (DFS) algorithm, the goal is to explore a graph starting from a given node, in this case, from node S, and find a path to a target node, G. The algorithm works by visiting a node, then recursively exploring its neighbors, diving as deep as possible into each path before backtracking and trying other branches. Starting from node S, DFS will explore each adjacent node, continuing down a branch until it reaches a dead end or the target node G. If a dead end is encountered, the algorithm backtracks to the most recent branching point and explores the next available path (like in node x in our implementation). This process continues until G is found, or all possible paths from S have been exhausted. DFS doesn't guarantee the shortest path, but it effectively finds any valid path from the start to the goal by exploring the graph deeply before considering alternative routes.

To implement the DFS algorithm, we will make two assumptions:

1. Nodes are traversed in lexicographical order, with Greek characters (Γ, Δ, Θ, Λ, Ξ, Π, Σ, Φ, Ω) having higher priority than English characters.
2. The algorithm always chooses the leftmost node when multiple options are available.



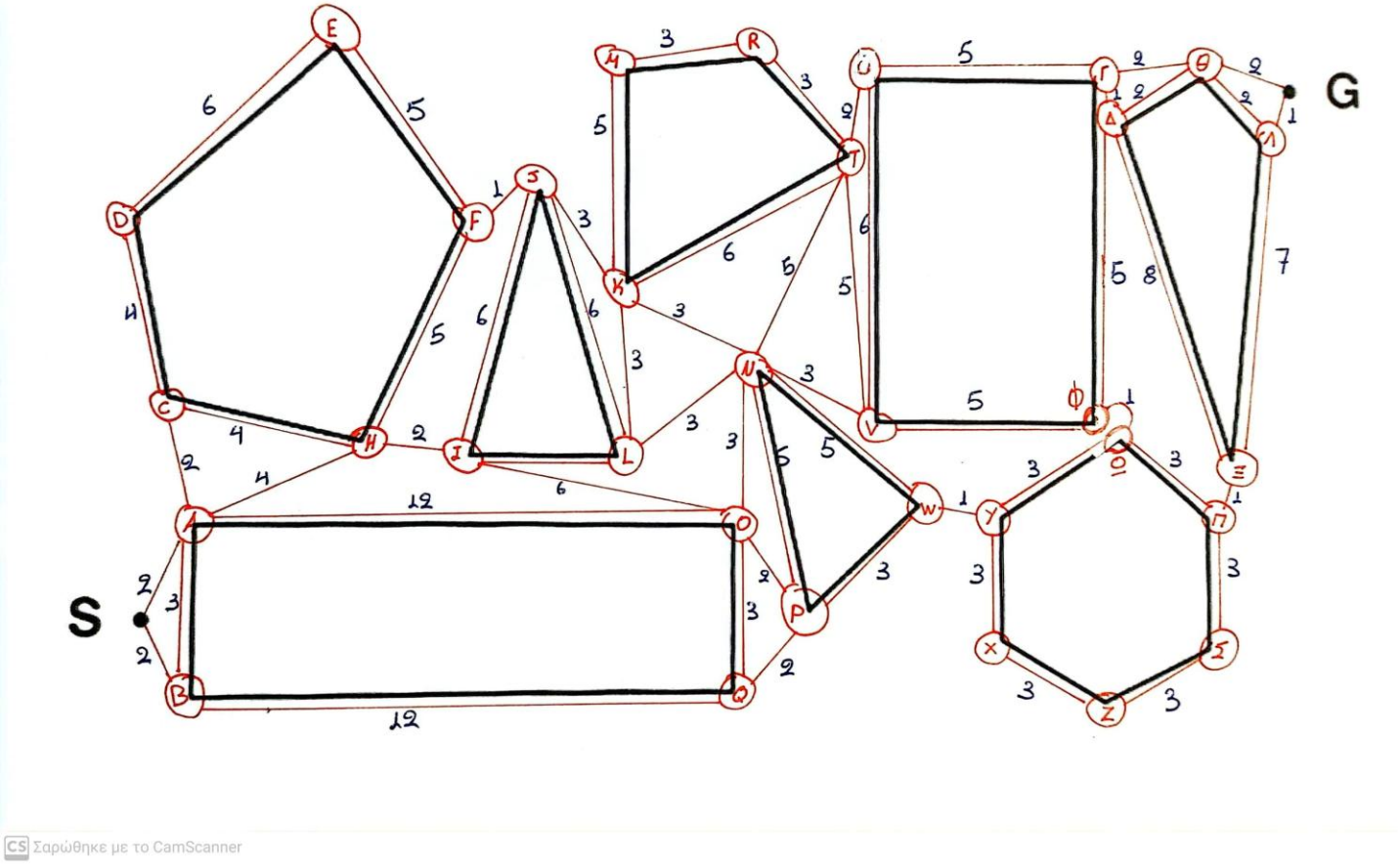
Expansion List
S
A
B
Q
O
I
H
C
D
E
F
J
K
L
N
P
W
Y
Ω
Π
Ξ
Λ
Θ
Γ
Δ



Φ
V
T
R
M
U
G

2.A.b Costs, heuristic and A* application of the problem

The following graph visually represents the nodes and connections, with the edges labeled by their respective costs. These costs reflect the approximate lengths of the straight-line segments between nodes, serving as the actual costs for traversing each edge.



No.	Node	H(n)	Actual cost	No.	Node	H(n)	Actual Cost
1.	S	21	30	19.	T	8	11
2.	A	20	28	20.	U	6	9
3.	B	22	31	21.	V	11	14
4.	C	20	28	22.	W	11	14
5.	D	23	32	23.	X	1	16
6.	E	20	26	24.	Y	10	13
7.	F	16	21	25.	Z	10	15
8.	H	17	24	26.	Γ	2	4
9.	I	16	22	27.	Δ	2	4
10.	J	15	20	28.	Θ	1	2
11.	K	13	17	29.	Λ	1	1
12.	L	14	19	30.	Ξ	7	8
13.	M	12	17	31.	Π	8	9
14.	N	12	16	32.	Σ	10	12
15.	O	14	19	33.	Φ	7	9
16.	P	13	17	34.	Ω	8	10
17.	Q	14	19	35.	G	0	0
18.	R	10	14				

The table presents heuristic estimates H(n) and actual costs for reaching a goal from various nodes in a graph. The heuristic function H(n) is admissible because it satisfies the condition $H(n) \leq$ Actual Cost(n) for all nodes, ensuring it never overestimates the true cost. For example, for node S, H(S)=21 and the actual cost is 30, so $H(S) \leq$ Actual Cost(S). This property guarantees that the heuristic is optimistic.

The A* algorithm calculates each path's value using $f(n)=g(n)+h(n)$, where g(n) is the cost from the starting node to the current node, and h(n) is the heuristic estimate of the remaining cost to reach the goal. This combination allows A* to prioritize the most promising nodes by expanding those with the lowest f(n) value first. When A* encounters a dead end or an inefficient path, it backtracks by re-evaluating nodes in the frontier, enabling it to avoid unproductive routes and continue searching for the best possible path. This systematic approach ensures that A* reliably finds the shortest path to the goal, provided the heuristic is admissible.

(Note: when we find 2 nodes with the same calculated cost, we randomly choose 1. Also, when we reach to a node we have in the frontier list, we delete the one with the worst cost)

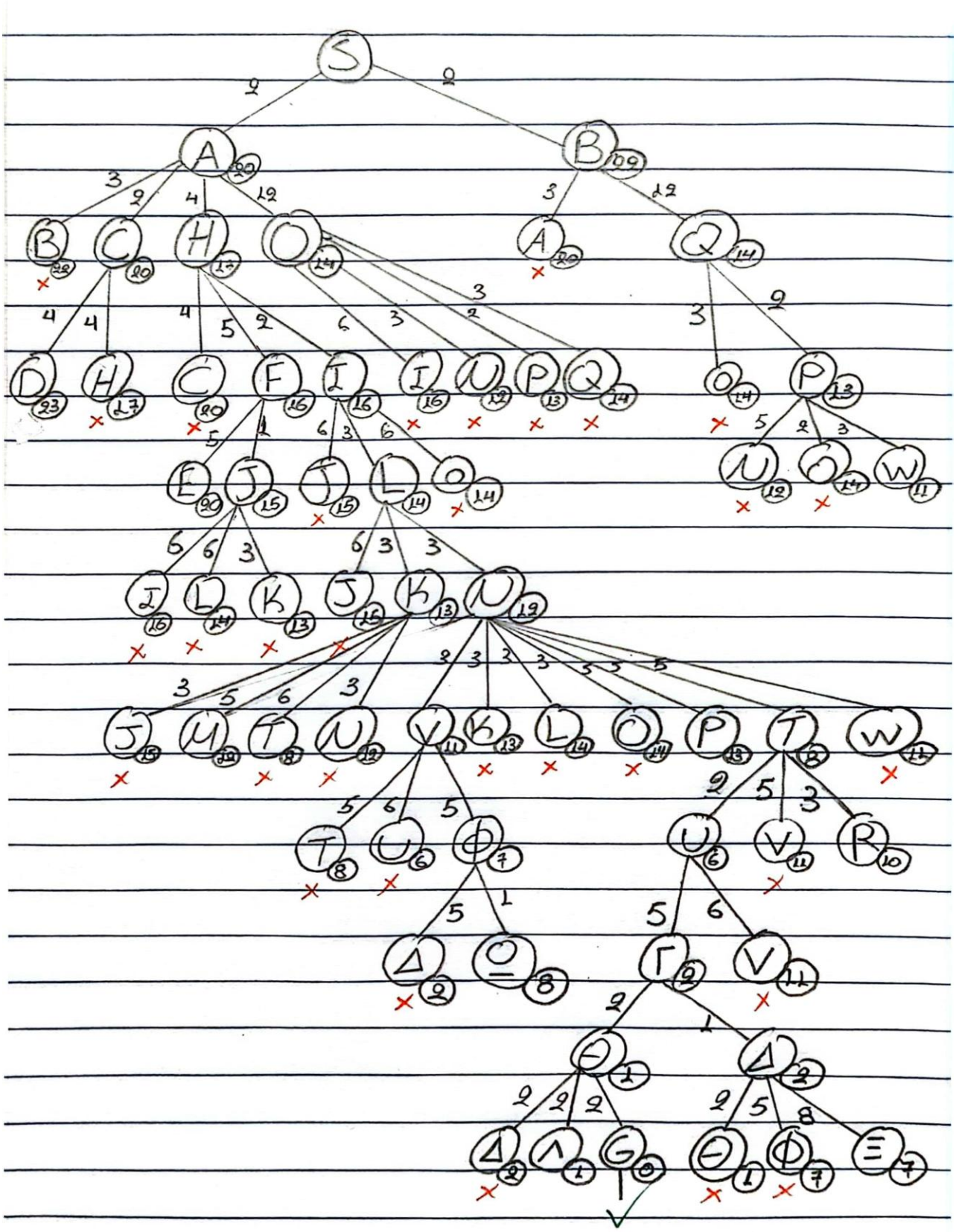
Steps	Frontier	Expansion List
Insert node S in frontier and expansion list	**S**	S
Extend node S, calculate costs and insert in the frontier, then select the node with the lower cost (A), and insert it in the expansion list and check if it is the target (false)	**S-A(22)**, S-B(24)	A
Extend node A and delete the duplicate nodes that have bigger cost (S-A-B) and insert the others in the frontier, calculate cost then select the node with the lower cost (H) and insert it in the expansion list and check if it is the target (false)	S-A-C(24), **S-A-H(23)**, S-A-O(28), S-B(24)	H
Extend node H, delete the duplicate nodes that have bigger cost (S-A-H-C) and insert the others in the frontier, calculate cost then select the node with the lower cost (I) and insert it in the expansion list and check if it is the target (false)	S-A-C(24), S-A-O(28), S-B(24), **S-A-H-I(24)**, S-A-H-F(27)	I
Extend node I, delete the duplicate nodes that have bigger costs (S-A-H-I-O) and insert the others in the frontier, calculate cost then select the node with the lower cost (B) and insert it in the expansion list and check if it is the target (false)	S-A-C(24), S-A-O(28), **S-B(24)**, S-A-H-F(27), S-A-H-I-J(29), S-A-H-I-L(25)	B
Extend node B, calculate cost, delete the nodes that have been in expansion list(S-B-A) and insert the others in the frontier, then select the node with the lower cost (C) and insert it in the expansion list and check if it is the target (false)	**S-A-C(24)**, S-A-O(28), S-A-H-F(27) S-A-H-I-J(29), S-A-H-I-L(25), S-B-Q(28)	C
Extend node B, calculate cost, delete the nodes that have been in expansion list before(S-A-C-H) and insert the others in the frontier, then select the node with the lower cost (L) and insert it in the expansion list and check if it is the target (false)	S-A-O(28), S-A-H-F(27) S-A-H-I-J(29), **S-A-H-I-L(25)**, S-B-Q(28), S-A-C-D(31),	L
Extend node L, delete the duplicate nodes that have bigger costs (S-A-H-I-L-J) and insert the others in the frontier, calculate cost then select the node with the lower cost (N) and	S-A-O(28), S-A-H-F(27) S-A-H-I-J(29), S-B-Q(28), S-A-C-D(31),	N

insert it in the expansion list and check if it is the target (false)	S-A-H-I-L-K(27), **S-A-H-I-L-N(26)**	
Extend node B, calculate cost, delete the nodes that have been in expansion list before and duplicate nodes that have bigger costs (S-A-H-I-L-N-K, S-A-H-I-L-N-L, S-A-H-I-L-N-O) and insert the others in the frontier, then select the node with the lower cost (T) and insert it in the expansion list and check if it is the target (false)	S-A-O(28), S-A-H-F(27) S-A-H-I-J(29), S-B-Q(28), S-A-C-D(31), S-A-H-I-L-K(27), S-A-H-I-L-N-P(32), **S-A-H-I-L-N-T(27)**, S-A-H-I-L-N-W(30), S-A-H-I-L-N-V(28)	T
Extend node T, calculate cost, delete duplicate nodes that have bigger costs (S-A-H-I-L-N-T-V) and insert the others in the frontier, then select the node with the lower cost (U) and insert it in the expansion list and check if it is the target (false)	S-A-O(28), S-A-H-F(27) S-A-H-I-J(29), S-B-Q(28), S-A-C-D(31), S-A-H-I-L-K(27), S-A-H-I-L-N-P(32), S-A-H-I-L-N-W(30), S-A-H-I-L-N-V(28), **S-A-H-I-L-N-T-U(27)**, S-A-H-I-L-N-T-R(32)	U
Extend node U, calculate cost, delete duplicate nodes that have bigger costs (S-A-H-I-L-N-T-U-V) and insert the others in the frontier, then select the node with the lower cost (T) and insert it in the expansion list and check if it is the target (false)	S-A-O(28), **S-A-H-F(27)** S-A-H-I-J(29), S-B-Q(28), S-A-C-D(31), S-A-H-I-L-K(27), S-A-H-I-L-N-P(32), S-A-H-I-L-N-W(30), S-A-H-I-L-N-V(28), S-A-H-I-L-N-T-U-Γ(28), S-A-H-I-L-N-T-R(32)	F
Extend node F, calculate cost, delete duplicate nodes that have bigger costs (S-A-H-I-J) and insert the others in the frontier, then select the node with the lower cost (T) and insert it in the expansion list and check if it is the target (false)	S-A-O(28), S-B-Q(28), S-A-C-D(31), **S-A-H-I-L-K(27)**, S-A-H-I-L-N-P(32), S-A-H-I-L-N-W(30), S-A-H-I-L-N-V(28), S-A-H-I-L-N-T-U-Γ(28), S-A-H-I-L-N-T-R(32), S-A-H-F-E(36), S-A-H-F-J(27)	K
Extend node K, calculate cost, delete the nodes that have been in expansion list before and duplicate nodes that have bigger costs (S-A-H-I-L-K-J, S-A-H-I-L-K-N, S-A-H-I-L-K-T) and insert the others in the frontier, then select the node with the lower cost (T) and	S-A-O(28), S-B-Q(28), S-A-C-D(31), **S-A-H-F-J(27)**, S-A-H-I-L-N-P(32), S-A-H-I-L-N-W(30), S-A-H-I-L-N-V(28), S-A-H-I-L-N-T-U-Γ(28), S-A-H-I-L-N-T-R(32),	J

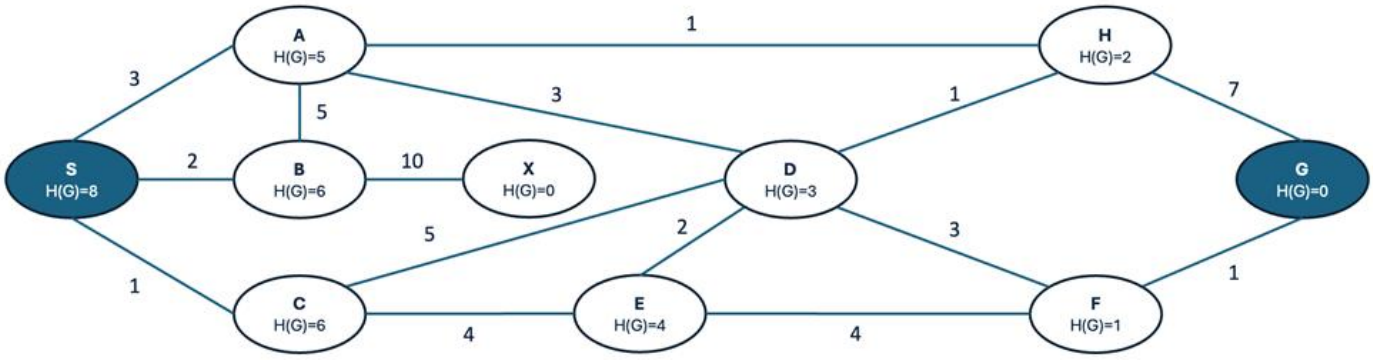
insert it in the expansion list and check if it is the target (false)	S-A-H-F-E(36), S-A-H-I-L-K-M(31)	
Extend node K, calculate cost, delete the nodes that have been in expansion list before and duplicate nodes that have bigger costs (S-A-H-F-J-I, S-A-H-F-J-K, S-A-H-F-L) and insert the others in the frontier, then select the node with the lower cost (Γ) and insert it in the expansion list and check if it is the target (false)	S-A-O(28), S-B-Q(28), S-A-C-D(31), S-A-H-F-E(36), S-A-H-I-L-N-P(32), S-A-H-I-L-N-W(30), S-A-H-I-L-N-V(28), **S-A-H-I-L-N-T-U-Γ(28)**, S-A-H-I-L-N-T-R(32), S-A-H-I-L-K-M(31)	Γ
Extend node Γ, calculate costs and insert in the frontier, then select the node with the lower cost (O), and insert it in the expansion list and check if it is the target (false)	**S-A-O(28)**, S-B-Q(28), S-A-C-D(31), S-A-H-F-E(36), S-A-H-I-L-N-P(32), S-A-H-I-L-N-W(30), S-A-H-I-L-N-V(28), S-A-H-I-L-N-T-R(32), S-A-H-I-L-K-M(31), S-A-H-I-L-N-T-U-Γ-Δ(29), S-A-H-I-L-N-T-U-Γ-Θ(29)	O
Extend node O, calculate cost, delete the nodes that have been in expansion list before and duplicate nodes that have bigger costs (S-A-O-I,S-A-O-N, S-A-O-Q) and insert the others in the frontier, then select the node with the lower cost (T) and insert it in the expansion list and check if it is the target (false)	S-A-O-P(29), **S-B-Q(28)**, S-A-C-D(31), S-A-H-F-E(36), S-A-H-I-L-N-P(32), S-A-H-I-L-N-W(30), S-A-H-I-L-N-V(28), S-A-H-I-L-N-T-R(32), S-A-H-I-L-K-M(31), S-A-H-I-L-N-T-U-Γ-Δ(29), S-A-H-I-L-N-T-U-Γ-Θ(29)	Q
Extend node Q, calculate cost, delete the duplicate node randomly (S-A-O-P) and insert the others in the frontier, then select the node with the lower cost (V) and insert it in the expansion list and check if it is the target (false)	S-B-Q-P(29), S-A-C-D(31), S-A-H-I-L-N-P(32), S-A-H-I-L-N-W(30), **S-A-H-I-L-N-V(28)**, S-A-H-I-L-N-T-R(32), S-A-H-F-E(36), S-A-H-I-L-K-M(31), S-A-H-I-L-N-T-U-Γ-Δ(29), S-A-H-I-L-N-T-U-Γ-Θ(29)	V
Extend node V, calculate cost, delete the nodes that have been in expansion list before (S-A-H-I-L-N-V-U, S-A-H-I-L-N-V-T) and insert the others in the frontier, then select the node with the lower cost (Θ) and insert it	S-B-Q-P(29), S-A-C-D(31), S-A-H-I-L-N-P(32), S-A-H-I-L-N-W(30), S-A-H-I-L-N-V-Φ(29), S-A-H-I-L-N-T-R(32),	Θ

in the expansion list and check if it is the target (false)	S-A-H-F-E(36), S-A-H-I-L-K-M(31), S-A-H-I-L-N-T-U-Γ-Δ(29), **S-A-H-I-L-N-T-U-Γ-Θ(29)**	
Extend node O, calculate cost, delete duplicate nodes that have bigger costs (S-A-H-I-L-N-T-U-Γ-Θ-Δ) and insert the others in the frontier, then select the node with the lower cost (T) and insert it in the expansion list and check if it is the target (false)	S-B-Q-P(29), S-A-C-D(31), **S-A-H-I-L-N-P(32)**, S-A-H-I-L-N-W(30), S-A-H-I-L-N-V-Φ(29), S-A-H-I-L-N-T-R(32), S-A-H-F-E(36), S-A-H-I-L-K-M(31), S-A-H-I-L-N-T-U-Γ-Δ(29), S-A-H-I-L-N-T-U-Γ-Θ-G(30), S-A-H-I-L-N-T-U-Γ-Θ-Λ(31)	P
Extend node P, calculate cost, delete the nodes that have been in expansion list before and the duplicate node randomly (S-B-Q-P-N,S-B-Q-P-O, S-A-H-I-L-N-W) and insert the others in the frontier, then select the node with the lower cost (Φ) and insert it in the expansion list and check if it is the target (false)	S-B-Q-P-W(30), S-A-C-D(31), S-A-H-I-L-N-P(32), **S-A-H-I-L-N-V-Φ(29)**, S-A-H-I-L-N-T-R(32), S-A-H-F-E(36), S-A-H-I-L-K-M(31), S-A-H-I-L-N-T-U-Γ-Δ(29), S-A-H-I-L-N-T-U-Γ-Θ-G(30), S-A-H-I-L-N-T-U-Γ-Θ-Λ(31)	Φ
Extend node Φ, calculate cost, delete the duplicate node randomly (S-A-H-I-L-N-V-Δ) and insert the others in the frontier, then select the node with the lower cost (V) and insert it in the expansion list and check if it is the target (false)	S-B-Q-P-W(30), S-A-C-D(31), S-A-H-I-L-N-P(32), S-A-H-I-L-N-V-Φ-Ω(31), S-A-H-I-L-N-T-R(32), S-A-H-F-E(36), S-A-H-I-L-K-M(31), **S-A-H-I-L-N-T-U-Γ-Δ(29)**, S-A-H-I-L-N-T-U-Γ-Θ-G(30), S-A-H-I-L-N-T-U-Γ-Θ-Λ(31)	Δ
Extend node V, calculate cost, delete the nodes that have been in expansion list before (S-A-H-I-L-N-T-U-Γ-Δ-Θ, S-A-H-I-L-N-T-U-Γ-Δ-Φ) and insert the others in the frontier, then select the node with the lower cost (G) and insert it in the expansion list and check if it is the target (true)	S-B-Q-P-W(30), S-A-C-D(31), S-A-H-I-L-N-P(32), S-A-H-I-L-N-V-Φ-Ω(31), S-A-H-I-L-N-T-R(32), S-A-H-F-E(36), S-A-H-I-L-K-M(31), S-A-H-I-L-N-T-U-Γ-Δ-Ξ(42), **S-A-H-I-L-N-T-U-Γ-Θ-G(30)**, S-A-H-I-L-N-T-U-Γ-Θ-Λ(31)	G

The algorithm exits and it has found the target		



2.B. SEARCH

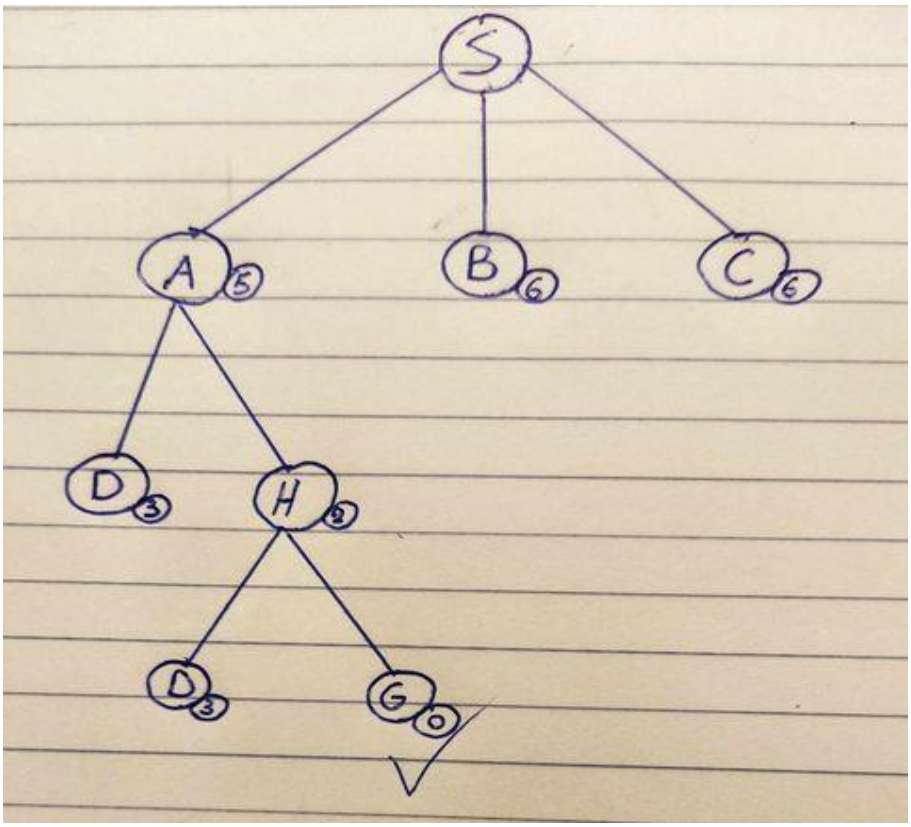


a. **Hill Climbing**: Hill Climbing algorithm takes account only the heuristic cost and not the actual cost to head to a node. Also, it only searches for the best cost without using backtracking and does not have a frontier or an expansion list. Though we can use a "frontier" so we can understand better the algorithm. Another problem we may face is that hill climbing algorithm may not always find the best solution or even worse we may not even reach our destination. On the other side, hill climbing algorithm is the most time efficient and memory-use efficient.

   In our case, the algorithm will find its target, though the cost is not the most efficient one: The path is S-A-H-G, and the final cost is 11.

Steps	"Frontier"
Initialize node S	**S**
Extend S and note heuristic	**S-A(5)**, S-B(6),S-C(6)
The node with the lower cost (S-A(5)) is not the target so we extend it and note the heuristic.	S-A-D(3), **S-A-H(2)**
The node with the lower cost is S-A-H(2) is not the target, so we must extend it and note the heuristic.	S-A-H-D(3), **S-A-H-G(0)**
The node with the lower cost is S-A-H-G(0) and is the target.	

And the hill climbing tree: (Note: The numbers that are located next to each node in a circle are the heuristic costs)

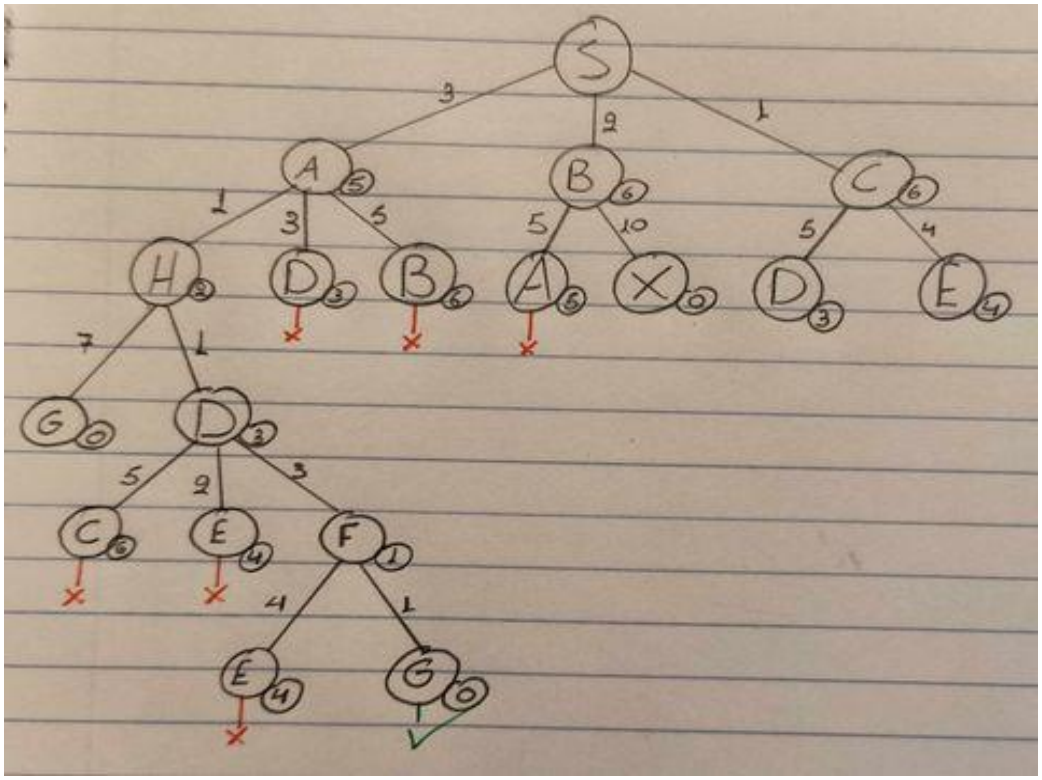


b. **A***: The A* algorithm calculates each path's value using f(n)=g(n)+h(n), where g(n) is the cost from the starting node to the current node, and h(n) is the heuristic estimate of the remaining cost to reach the goal. This combination allows A* to prioritize the most promising nodes by expanding those with the lowest f(n) value first. When A* encounters a dead end or an inefficient path, it backtracks by re-evaluating nodes in the frontier, enabling it to avoid unproductive routes and continue searching for the best possible path. This systematic approach ensures that A* reliably finds the shortest path to the goal, provided the heuristic is admissible.
(Note: when we find 2 nodes with the same calculated cost, we randomly choose 1. Also, when we reach to a node we have passed before, we choose the one with the best cost)

In our case, A* algorithm will find the shortest path which is S-A-H-D-F-G, and the final cost will be 9

Steps	Frontier	Expansion List

Insert node S in frontier and expansion list	**S**	S
Extend node S and calculate costs, then select the node with the lower cost (C), and insert it in the expansion list and check if it is the target (false)	S-A(8), S-B(8), **S-C(7)**	C
Extend node C, calculate costs, then select the node with the lower cost (A) and insert it in the expansion list and check if it is the target (false)	**S-A(8)**, S-B(8), S-C-D(11), S-C-E(9)	A
Extend node D and delete the duplicate nodes that have bigger cost (S-A-D, S-A-B), calculate cost then select the node with the lower cost (H) and insert it in the expansion list and check if it is the target (false)	S-A-B(14), S-A-D(9), **S-A-H(6)**, S-C-E(9)	H
Extend, node H, calculate costs, then select the node with the lower cost (D) and insert it in the expansion list and check if it is the target (false)	S-A-B(14), S-A-D(9), **S-A-H-D(8)**, S-A-H-G(11), S-C-E(9)	D
Extend node D and delete the duplicate nodes that have bigger cost (S-A-H-D-C, S-A-H-D-E), calculate cost then select the node with the lower cost (F) and insert it in the expansion list and check if it is the target (false)	S-A-B(14), S-A-H-D-E(11), **S-A-D-F(9)**, S-A-H-G(11), S-C-E(9)	F
Extend node F and delete the duplicate nodes that have bigger cost (S-A-H-D-F-E), calculate cost then select the node with the lower cost (G) and insert it in the expansion list and check if it is the target (True)	S-A-B(14), S-A-H-D-E(11), **S-A-D-F-G(9)**, S-A-H-G(11), S-C-E(9)	G
The algorithm exits and it has found the target		

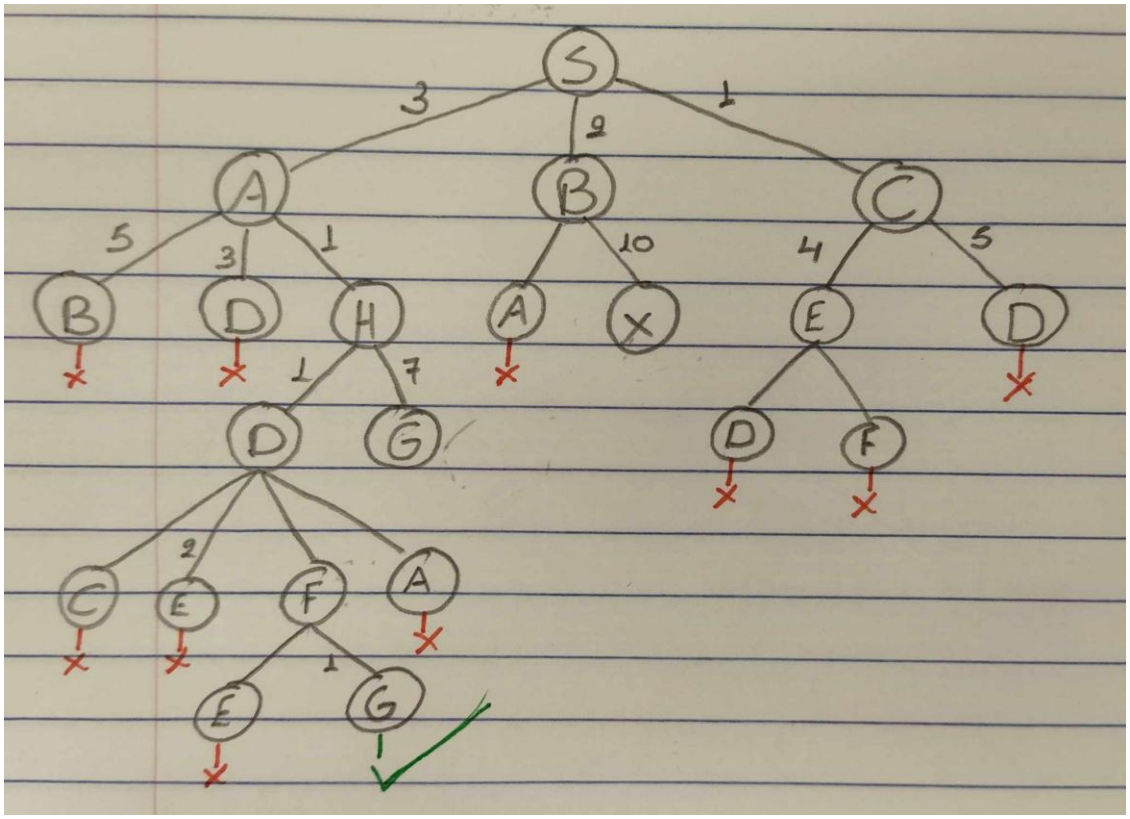


c. **Uniform Cost**: Uniform cost algorithm takes into consideration only the actual cost from one node to another. It searches for the best path by heading to the node with the lower cost. Though just by heading each time to the lower cost node does not mean that the path the algorithm finds is the most efficient one, if the costs do not get lower when we are heading from one node to another. Also, UCS uses a frontier and expansion list to ensure that we don't have any circles and not have 2 similar nodes.

In our case, the algorithm finds the best path, which is S-A-H-D-F-G, and the final cost will be 9.

Steps	Frontier	Expansion List
Initialize node S and insert in expansion List	**S**	S
Extend node S, note them in the frontier with their costs (S-A(3), S-B(2), S-C(1)), select the node with the lower cost, insert it in the expansion list and check if it is the target (False)	S-A(3), S-B(2), **S-C(1)**	C
Extend node C, note them in the frontier with their costs (S-A(3), S-B(2), S-C-E(5),S-C-D(6)), select the	S-A(3), **S-B(2)**,	B

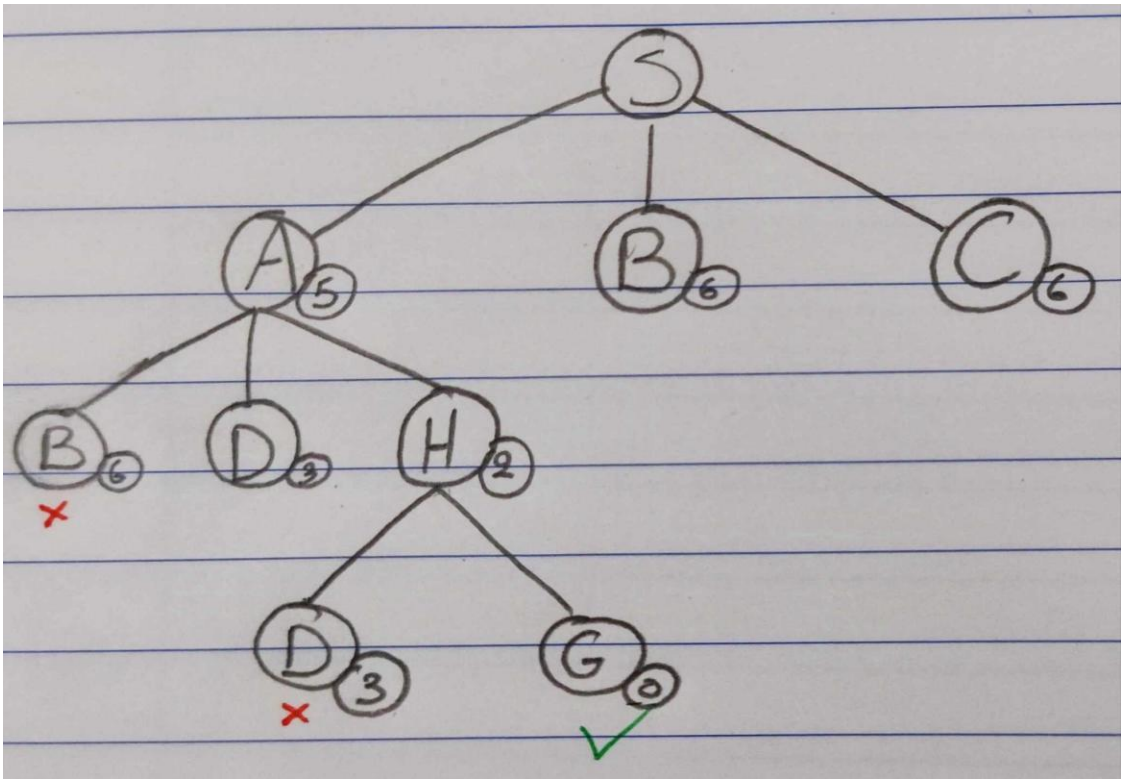
node with the lower cost, insert it in the expansion list and check if it is the target (False)	S-C-E(5), S-C-D(6)	
Extend node B, note them in the frontier with their costs (S-B-A(7), S-B-X(12)) though we have a better path to A so no need to write it in frontier, select the node with the lower cost, insert it in the expansion list and check if it is the target (False)	**S-A(3)**, S-B-X(12), S-C-E(5), S-C-D(6)	A
Extend node A, note them in the frontier with their costs (S-A-B(8),S-A-D(6),S-A-H(4)) though we have a similar or better path to B and D, so no need to write it in frontier, select the node with the lower cost, insert it in the expansion list and check if it is the target (False)	**S-A-H(4)**, S-B-X(12), S-C-E(5), S-C-D(6)	H
Extend node H, note them in the frontier with their costs (S-A-H-D(5), S-A-H-G(11)), we have found a better path to D so we delete the previous one, select the node with the lower cost, insert it in the expansion list and check if it is the target (False)	**S-A-H-D(5)**, S-A-H-G(11), S-B-X(12), S-C-E(5)	D
Extend node D, note them in the frontier with their costs (S-A-H-D-C(10), S-A-H-D-E(7), S-A-H-D-F(8),S-A-H-D-A(8)), we have found a better path to E and C and A are in the expansion list so no need to be in the frontier, select the node with the lower cost, insert it in the expansion list and check if it is the target (False)	S-A-H-D-F(8), S-A-H-G(11), S-B-X(12), **S-C-E(5)**	E
Extend node E, note them in the frontier with their costs (S-C-E-D(7),S-C-E-F(9)),we have found a better path to F and D is in the expansion list so no need to be in the frontier, select the node with the lower cost, insert it in the expansion list and check if it is the target (False)	**S-A-H-D-F(8)**, S-A-H-G(11), S-B-X(12)	F
Extend node F, note them in the frontier with their costs (S-A-H-D-F-E(12),S-A-H-D-F-G(9)), E is in the expansion list and we now found a better path to G so no need to be in the frontier, select the node with the lower cost, insert it in the expansion list and check if it is the target (True)	**S-A-H-D-F-G(8)**, S-B-X(12)	G
The algorithm exits and it has found the target		



d. **Best-First Search** is an efficient algorithm that improves upon Hill Climbing by using a structured approach to exploration. It uses a frontier, which stores all unexplored paths, and an expansion list, which keeps track of visited nodes to avoid cycles and duplicate paths. Unlike Hill Climbing, which only considers the immediate neighbors of the current node, Best-First Search evaluates and prioritizes all paths in the frontier based on a heuristic cost, selecting the most promising one. This approach allows it to backtrack and explore alternative paths if necessary, making it more robust and less prone to issues like getting stuck in local maxima or dead ends. By maintaining and managing its frontier, Best-First Search ensures a broader and more efficient search for the target.
In our case, the algorithm will find its target, though the cost is not the most efficient one: The path is S-A-H-G, and the final cost is 11.

Steps	Frontier	Expansion List
Initialize node S and insert in expansion list	S	S
Extend S and note heuristic, insert them in the frontier, select the node with the lower cost, insert it in expansion list and check if it is the target (False)	**S-A(5)**, S-B(6), S-C(6)	A
Extend A and note heuristic, insert them in the frontier, delete duplicate nodes (S-A-B(6)*), select the node with the lower cost, insert it in expansion list and check if it is the target (False)	**S-A-H(2)**, S-A-D(3), S-B(6), S-C(6)	H
Extend H and note heuristic, insert them in the frontier, delete duplicate nodes (S-A-H-D(3)*), select the node with the lower cost, insert it in expansion list and check if it is the target (False)	**S-A-H-G(0)**, S-A-D(3), S-B(6), S-C(6)	G
The algorithm exits and it has found the target		

*When we have duplicates usually, we keep the first node, though it depends in the execution of the code.

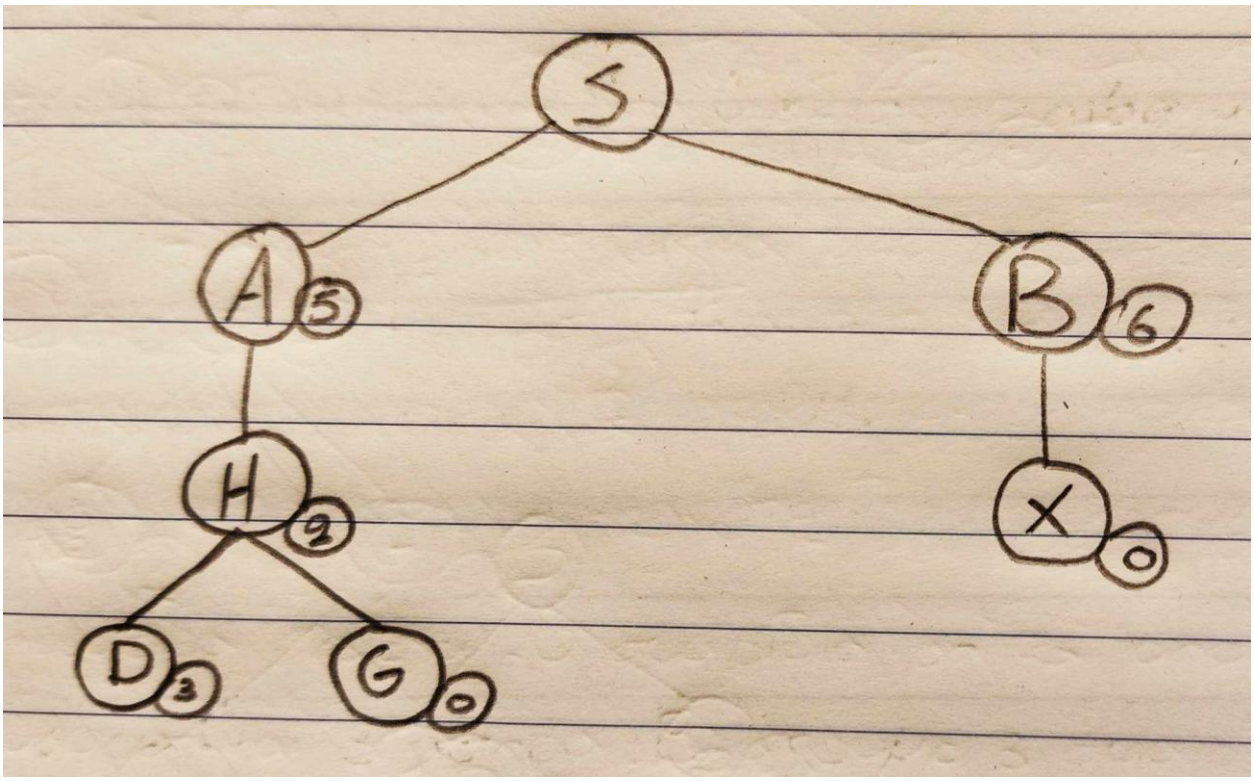


e. **Beam Search (w=2)**: Beam search algorithm uses the heuristic cost to find the shortest path. From the expanded set, the selection step selects only the top w=2 nodes with the best heuristic values. Also, it doesn't use a frontier or an expansion list, so it

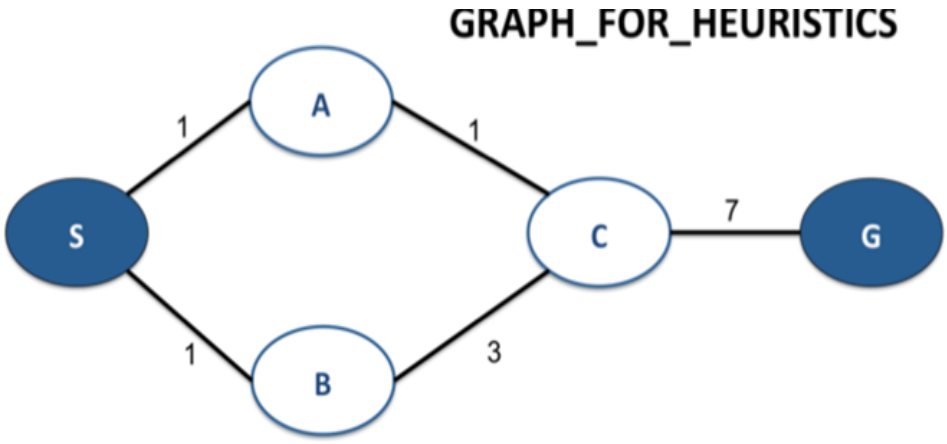
doesn't track the possible paths but only the 2 available nodes. Though, only to understand better the algorithm we can use a "frontier".

In our case, the algorithm will find its target, though the cost is not the most efficient one: The path is S-A-H-G, and the final cost is 11.

Steps	"Frontier"
Initialize node S and insert in expansion list	S
Extend node S, expand it by selecting the w=2 nodes with the lower cost, S-A(5) and S-B(6)	S-A(5), S-B(6)
The nodes with the lower cost (S-A(5), S-B(6)) are not the target so we expand them by selecting the 2 nodes with the lower cost, S-A-H(2) and S-B-X(0).	S-A-H(2), S-B-X(0)
The nodes with the lower cost (S-A-H(2) and S-B-X(0)) are not the target so we expand them by selecting the 2 nodes with the lower cost, S-A-H-G(2) and S-A-H-D (3).	S-A-H-D(3), S-A-H-G(0)
The node S-A-H-G(0) is our target so the algorithms exits	



3.Heuristic Functions



a. ***Find an admissible but inconsistent heuristic function.***

An admissible heuristic is one that never overestimates the actual cost to the goal (H(n)≤ actual cost) but may not satisfy the triangle inequality (H(n)-H(m) ≤cost(n, m)) which makes it inconsistent.

Node	H(n)	Cost to node G
S	4	9
A	3	8
B	2	10
C	1	7
G	0	0

We notice that in each node, H(n) <= actual cost. So, our heuristic function is admissible.

Though, it is inconsistent. For example, from node S to B, H(S)-H(B)-4-2=2 > cost(S, B)=1.

b. ***Find an admissible but inconsistent heuristic where A* fails to find the optimal solution.***

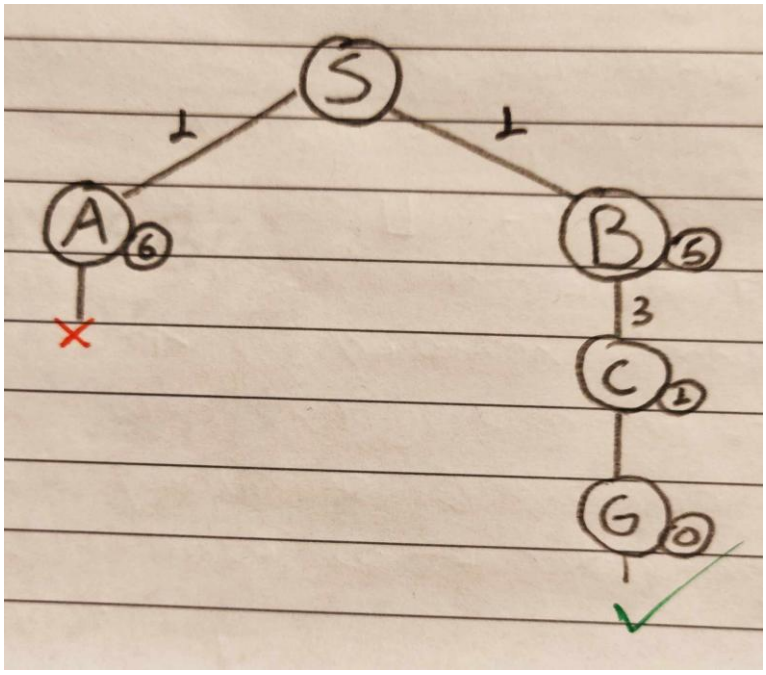
Node	H(n)	Cost to node G
S	5	9
A	6	8
B	1	10
C	1	7
G	0	0

We notice that in each node, H(n) <= actual cost. So, our heuristic function is admissible

Though, if we try A* algorithm, it exits finding that the best path is S-B-C-G and the best cost is 11,

Frontier	Expansion List
S	S
S-A(7), S-B(6)	B
S-A(7), S-B-C(5)	C
S-A(7), S-B-C-G(11)	A
S-B-C-G(11)	G

which is false, because the path S-A-C-G has a cost of 9.

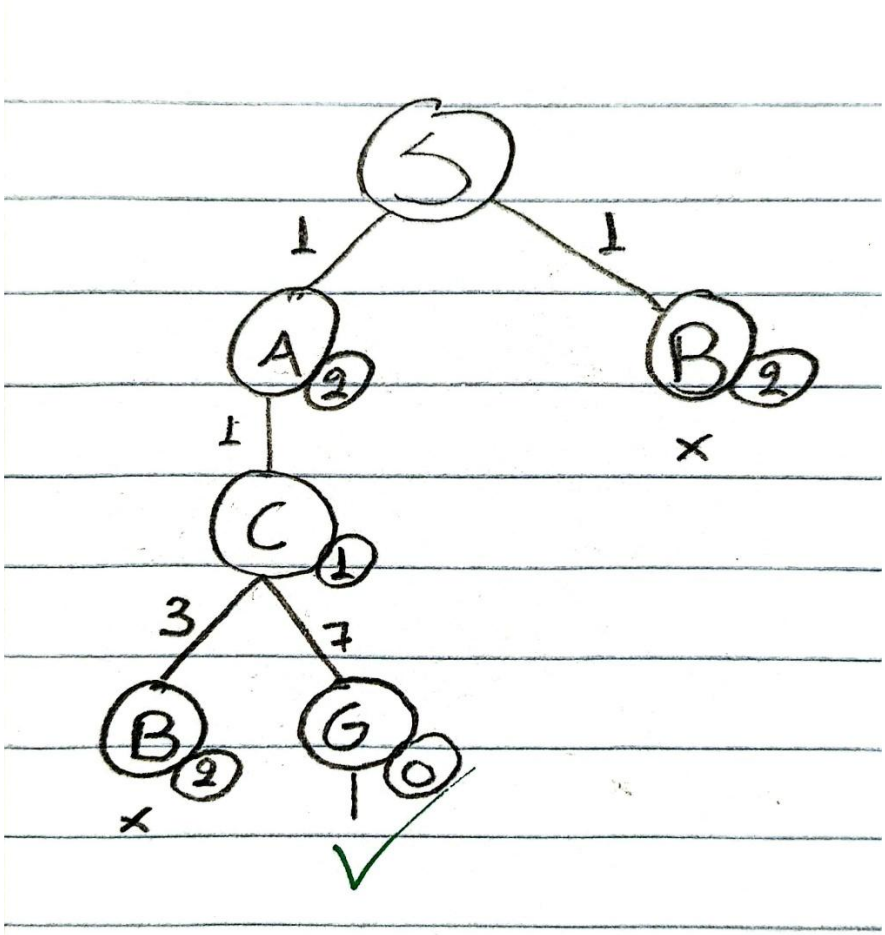


c. *Find an admissible but not consistent heuristic that leads A* to the optimal solution*

To ensure that while the heuristic is inconsistent, it does not misguide A* into choosing suboptimal paths, consider the following example H(S)=4, H(A)=2, H(B)=2, (C)=1, and (G)=0. This heuristic is admissible because H(n)≤actual cost to goal H(n)≤actual cost to goal for all nodes. For example, H(S)=4≤7 (actual cost via A→C→G). However, it is inconsistent because it violates the triangle inequality. For instance, H(A)-H(C)=4-1=3>cost(A,C)=2. Despite the inconsistency, A* will still find the optimal solution S→A→C→G, as no suboptimal paths (e.g., through B) are overly prioritized due to this heuristic. This ensures that the inconsistency does not mislead the search process.

Node	H(n)	Cost to node G
S	4	9
A	2	8
B	2	10

C	1	7
G	0	0



Steps	Frontier	Expansion List
Insert node S	S	S
Extend S, note to the frontier, select the node with the lower cost, insert it in the frontier list and check if target (False)	**S-A(3)**, S-B(3)	A
Extend A, note to the frontier, select the node with the lower cost, insert it in the frontier list and check if target (False)	**S-A-C(3)**, S-B(3)	C
Extend C, note to the frontier, select the node with the lower cost, insert it in the frontier list and check if target (False)	S-A-C-G(9), **S-B(3)**	B
Extend B, insert in the frontier the nodes that have not been mentioned in the exp. list (None), select the node with the lower cost, insert it in the frontier list and check if target (True)	**S-A-C-G(9)**	G